SkyBook

An airline reservations platform, built end to end

Recruiter edition — an 11-page summary of the full engineering case study

Author: Praveenreddy Somireddy

At a glance

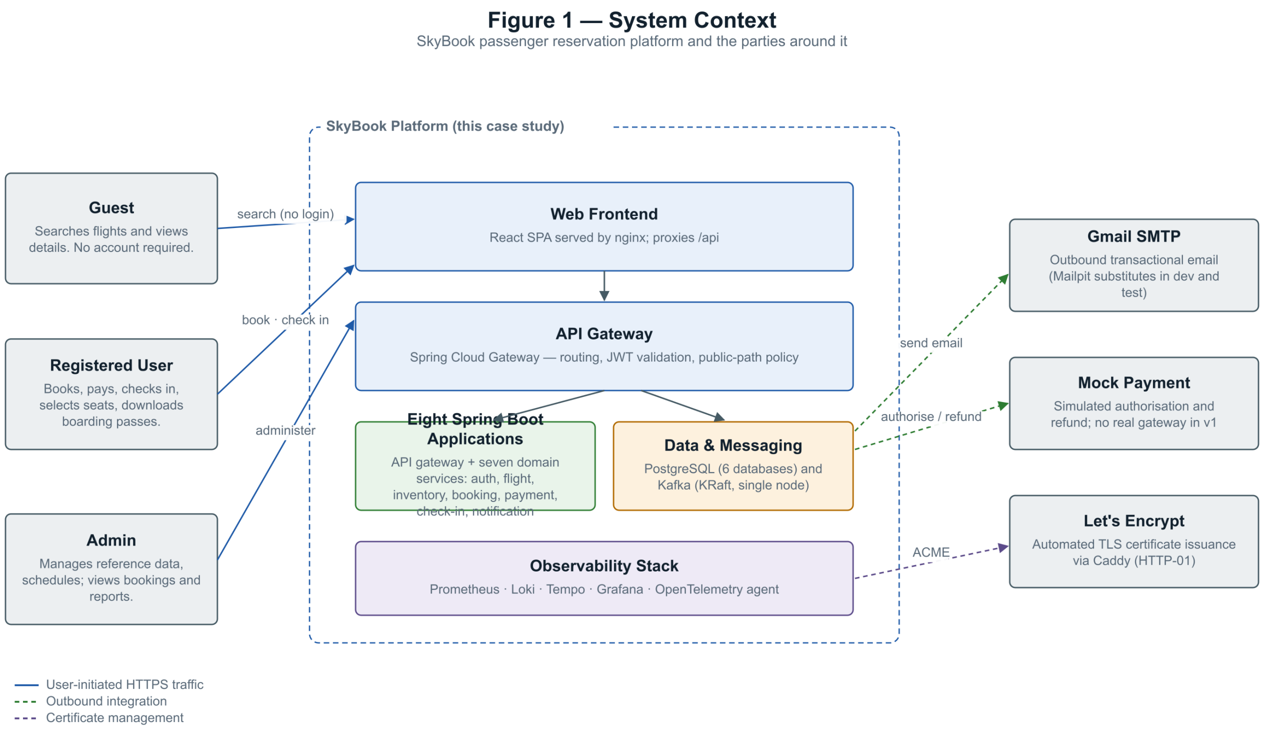
Item	Detail
What	A working airline-reservation platform: anonymous search, real seat maps, payment, confirmation email, check-in, scannable boarding pass, cancellations with fare-rule refunds, an admin console
Shape	Eight Spring Boot applications (one API gateway, seven domain services), React 19 web client, PostgreSQL, Kafka, full observability
Built	35 calendar days of designed increments, then a measured quality and release-engineering revision
Quality	SonarCloud gate PASSING: 91.2% coverage, 0 bugs, 0 vulnerabilities, security and reliability rating A
Tests	1,722 backend executions + 55 frontend + a 24-test end-to-end certification that races real buyers for one seat - zero failures at the evidence commit
Environments	A real promotion ladder: LOCAL > DEV > SIT > QA > PERF > UAT > STAGING > PROD, plus a scheduled weekly disaster-recovery drill - the first full destroy-and-restore cycle passed on 4 August 2026
Deployment	One cloud VM, 18 containers, automatic TLS - promoted by image digest, never rebuilt on the way to production
Author	Praveenreddy Somireddy - solo: requirements, architecture, backend, frontend, testing, CI/CD, cloud deployment and documentation, every decision and merge owned end to end

[Repository: github.com/praveencloudlab/skybook](https://github.com/praveencloudlab/skybook)

[Quality dashboard: sonarcloud.io/project/overview?id=praveencloudlab_skybook](https://sonarcloud.io/project/overview?id=praveencloudlab_skybook)

Disclaimer: SkyBook is a personal portfolio project - not a real airline, affiliated with no carrier. Airline-style codes in seed data are illustration only; payments are simulated server-side and no cardholder data exists; all passenger records are synthetic.

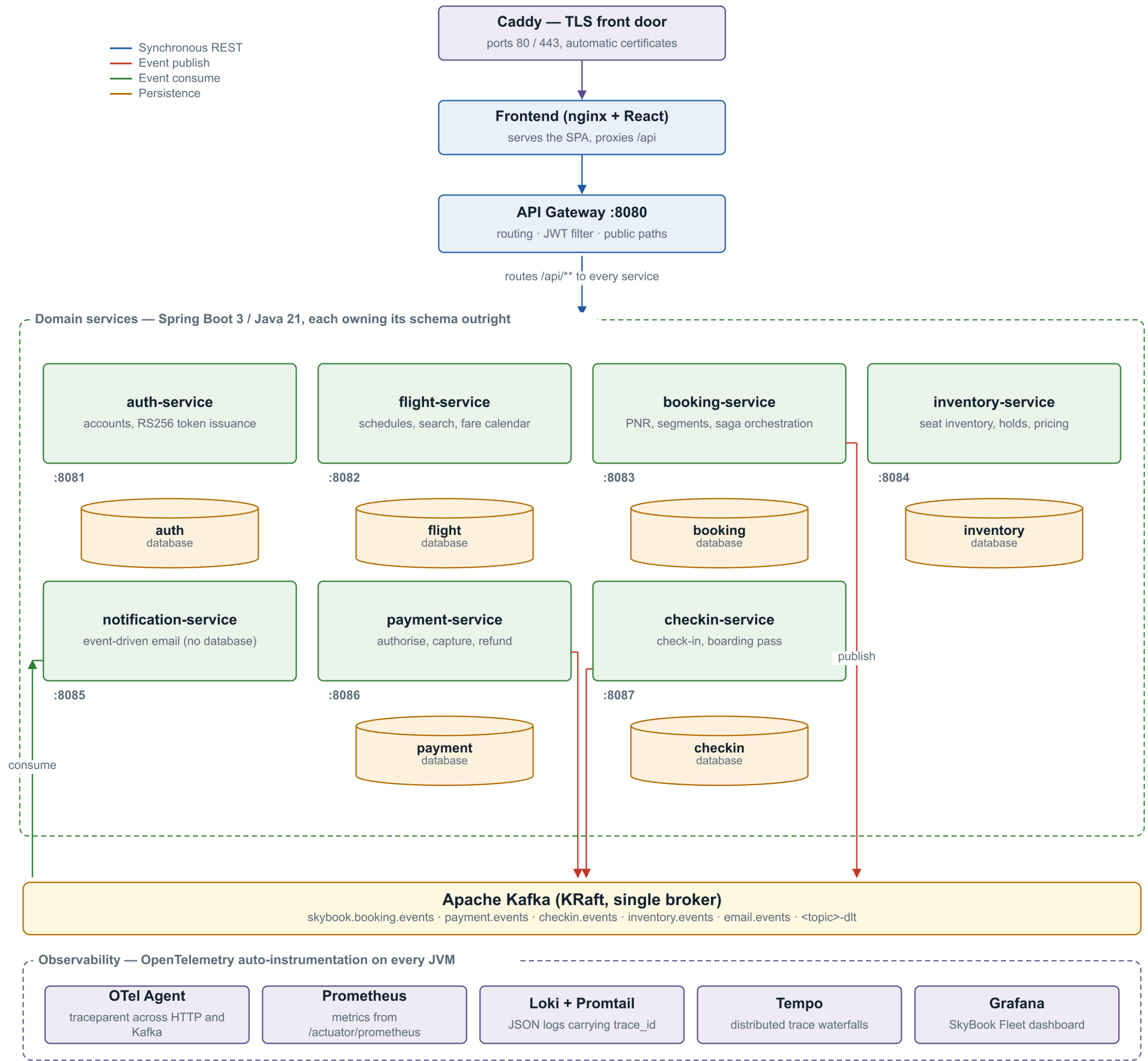
The system



The context: a public shopping surface, an authenticated booking core, and operations behind an admin role.

Figure 2 — Microservice Architecture

Runtime components, synchronous routes and asynchronous event flow



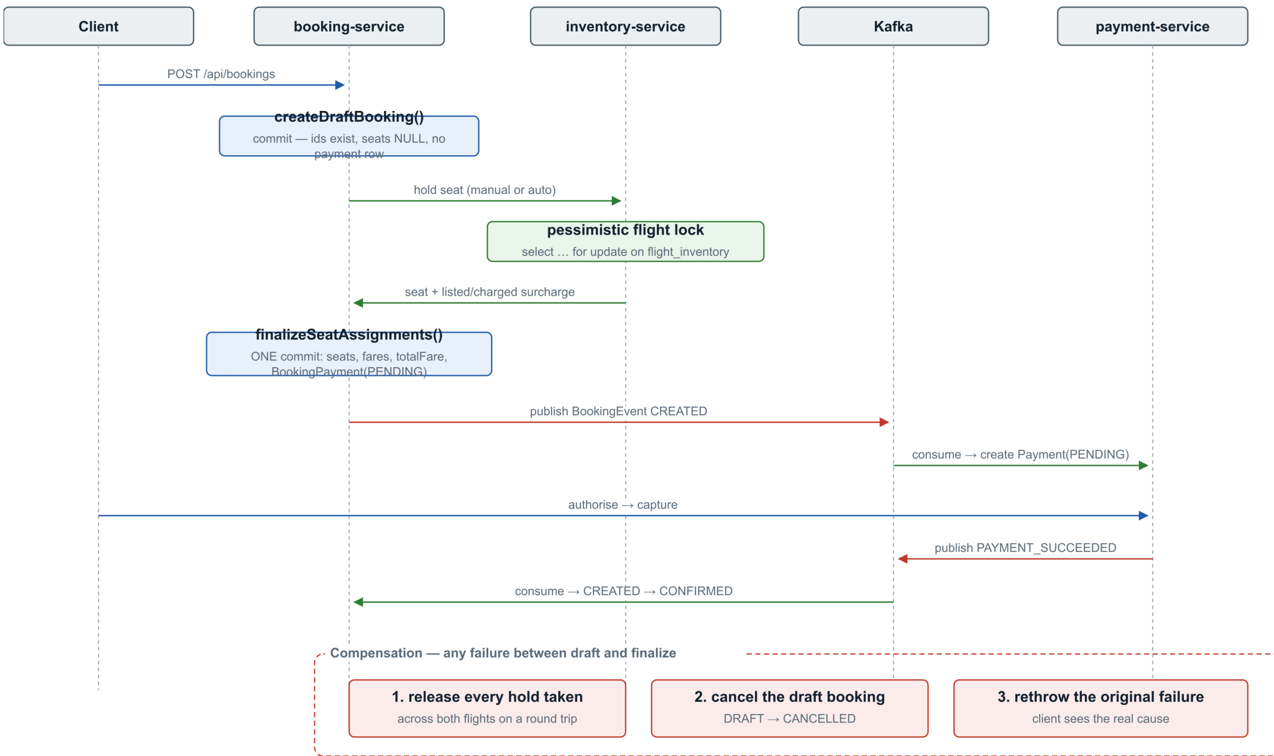
Eight applications over one PostgreSQL instance hosting six service-owned databases, and one event bus. Every service owns its data; nothing reaches another service's tables.

The hard part: money moves asynchronously

A booking is not confirmed when the payment call returns - it is confirmed when payment-service says so over Kafka and booking-service reacts. The saga below is the platform's spine, and the double-sell race (two real buyers, one seat, concurrent checkout) is certified end to end every night and on every promotion.

Figure 5 — Booking Saga

draft → hold → finalize, with compensation on any failure



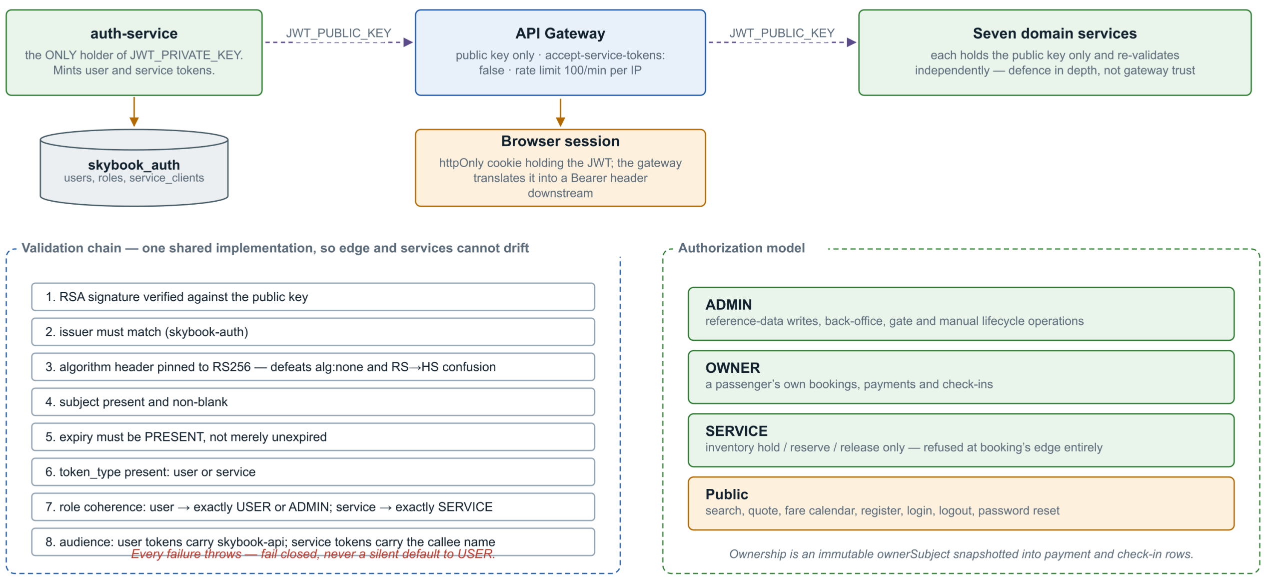
The facade is deliberately NOT @Transactional: every external call happens outside a database transaction, so no connection is held across network I/O.

The booking saga: draft, hold, pay, confirm - with compensations, not distributed locks.

Security, stated honestly

Figure 10 — Authentication and Authorization

RS256 asymmetric signing: one service mints, everyone verifies



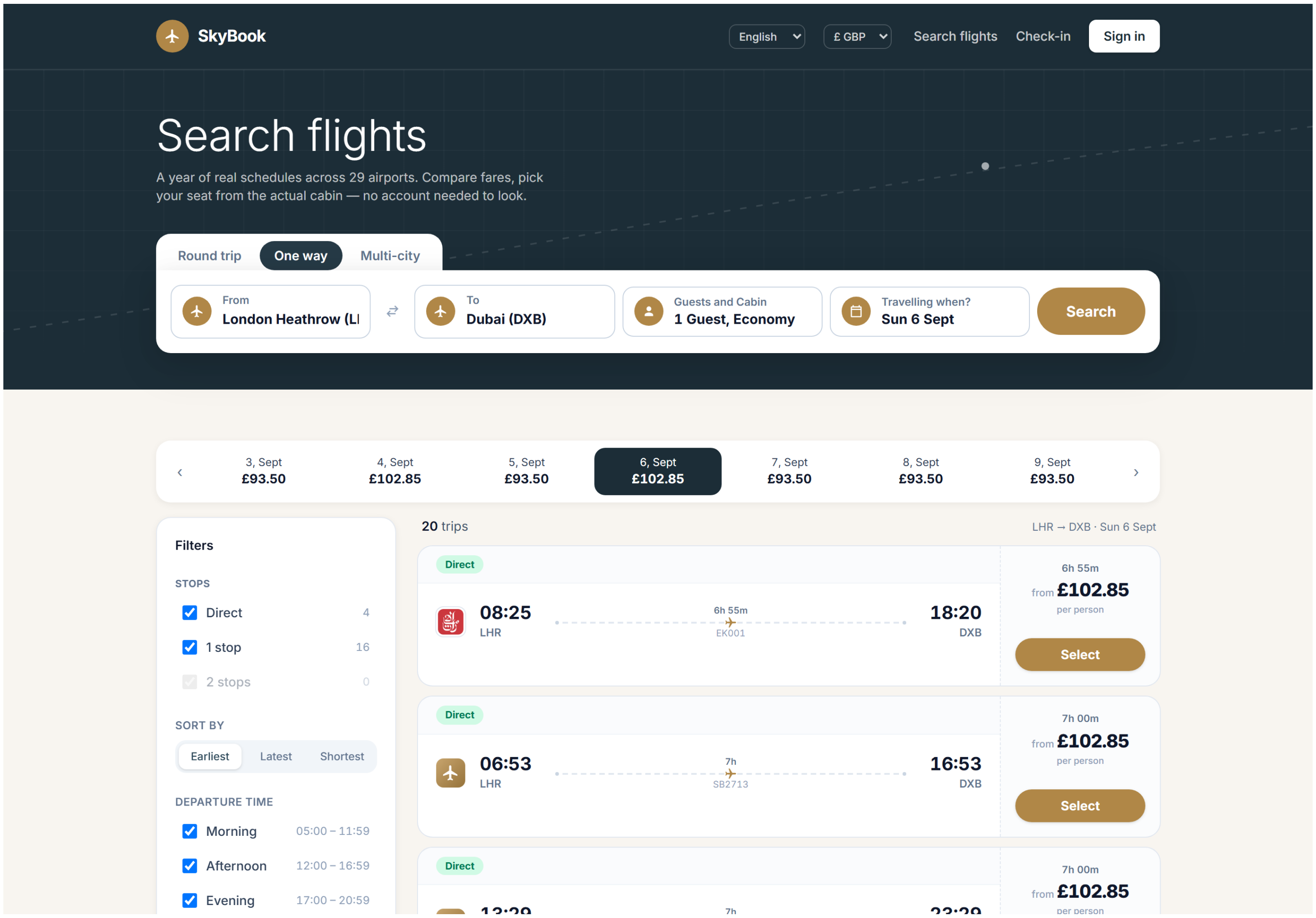
RS256 sessions, per-service credentials, an httpOnly cookie the gateway exchanges for a Bearer header - and the gaps stated next to the controls.

The eight CSRF findings SonarCloud raised were closed with written justifications in code - the cookie is consumed at the gateway, SameSite=Lax covers the SPA, and all 48 GET endpoints were audited as read-only. A leaked mail credential early in the project was revoked, purged from history, and is recorded in the study as an incident, not hidden.

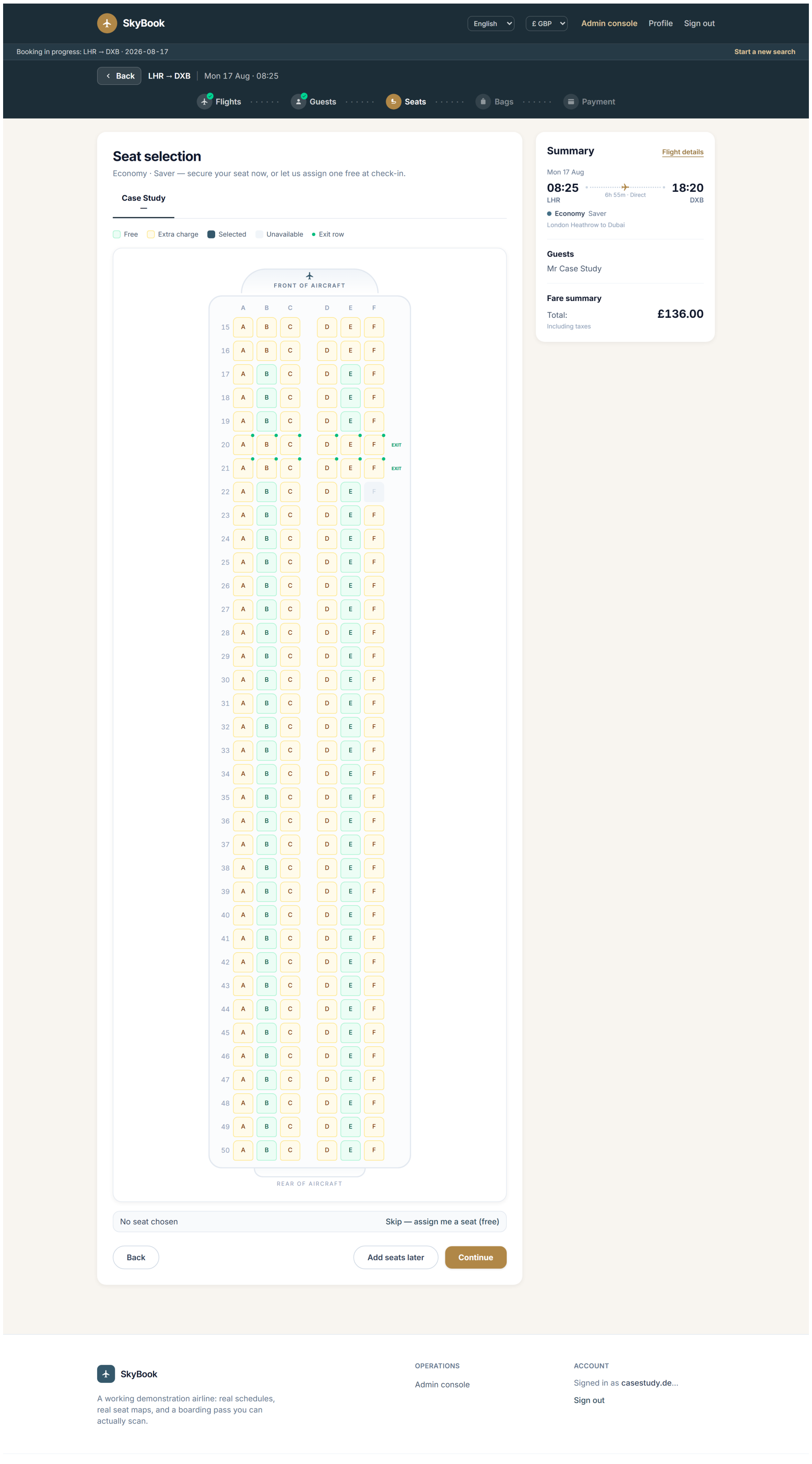
Evidence, not adjectives

Claim	Proof
1,722 backend test executions, zero failures	mvn clean verify at commit 96e24b2, 4 Aug 2026 - per-module table in the full study sums to exactly this
91.2% coverage, gate passing	Public SonarCloud dashboard, measured on every push
The journey certifies	24 e2e tests through the gateway only: booking, payment, check-in, boarding pass, double-sell race, email into a real sink, one trace across the Kafka hop
Fast enough under load	k6: 3,405/3,405 checks, p95 search 60 ms, quote 26 ms, 0.00% errors - thresholds gate every promotion
Backups actually restore	Scheduled weekly drill - first full cycle passed 4 Aug 2026: back up, destroy the database volume, restore, verify 1,336,748 rows against the manifest
Same artifact everywhere	Images built once per commit, scanned, published as multi-arch digests; every environment from DEV to PROD runs those exact bytes

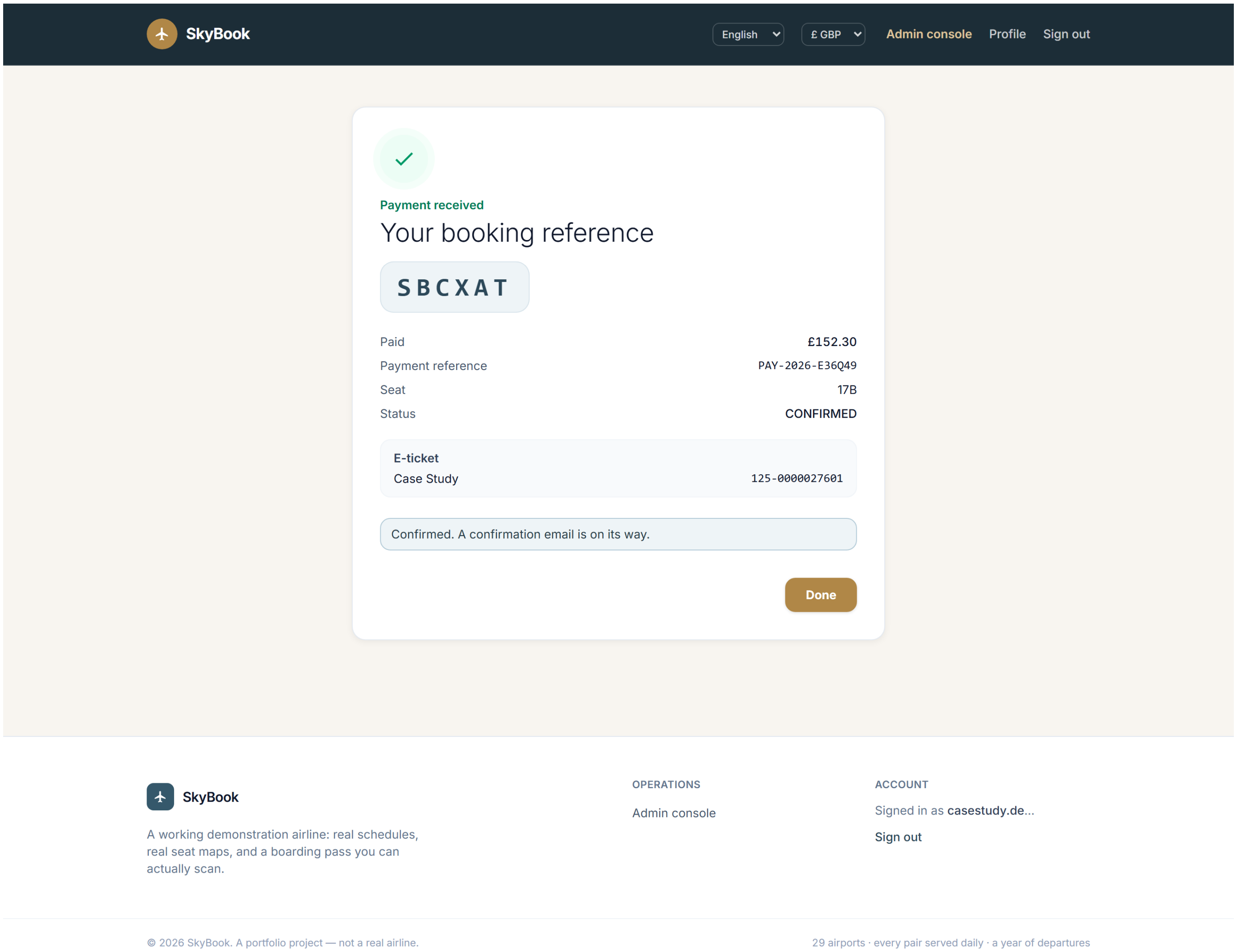
The product



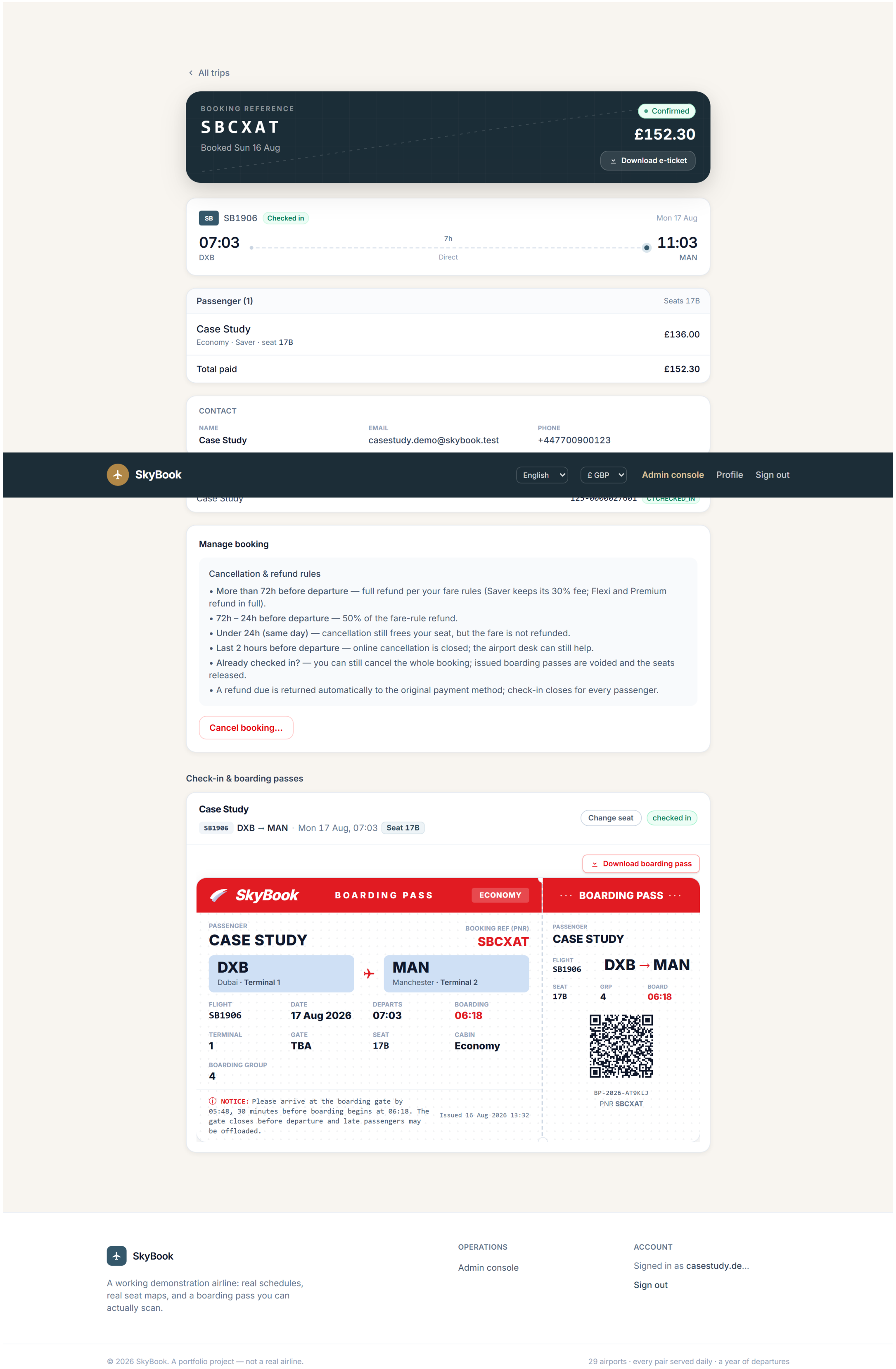
Search: a seven-day price strip and one card per itinerary - priced by the same endpoint checkout will use.



Seat selection from the real cabin: paid rows priced per seat, held seats disabled, free auto-assign.



The moment three services agree: record locator, paid amount, trip saved.



The boarding pass - the QR is real and the gate-operations console verifies it.

Five things that went wrong, on purpose in print

Incident	Lesson
A transactional method silently ran with no transaction (Spring self-invocation)	Proxies do not apply to internal calls; the nightly schedule generator ran unprotected until static analysis flagged it
Every cross-timezone arrival displayed hours wrong - durations looked right by accident	Two wall clocks in different zones subtract to nonsense; fixing it touched generation, email, search, 414,679 rows and the screenshots
A round-trip cancellation could refund partially and leave the booking looking live	The expansion set and the remaining set were computed from different keys; pinned by tests that fail on the old code
The environment ladder's first ever run refused a bad promotion, then caught its own seeding bug	Gates that only pass are decoration; these blocked, and the fixes are commits
A test demanded zero collisions from a generator whose own name promised "rare"	The birthday paradox made CI red one run in twenty-two; assert what the code promises, not more

Current limits, stated: no transactional outbox (a post-commit publish failure loses the event - top of the roadmap), no sustained capacity test (the k6 gate catches regressions, not limits), single-VM deployment with RPO 24h / RTO ~30min, and a browser UI suite not yet wired into CI. The full study gives every gap its reasons.

The full engineering edition covers the architecture, the saga, pricing and refunds, check-in, the delivery history commit by commit, testing, defects and lessons, the data model, and six appendices - traceability, API contracts, security operations, screens, roadmap, glossary.